

Bài giảng W28 - Thứ 6 sáng: Methods và **init**

Mục tiêu bài học

- Hiểu và sử dụng methods trong class
- Nắm vững constructor **init**
- Phân biệt giữa class variables và instance variables
- Chuẩn bị kiến thức cho mini project Bank Account System

Kiến thức đã học (từ các buổi trước)

- Variables, Data types, Input/Output
- Operators, If/else statements
- Loops (for, while)
- Lists, Dictionaries, Tuples
- Functions cơ bản
- OOP concepts, Classes và Objects cơ bản

1. Ôn tập Class và Object cơ bản

Từ buổi học trước, chúng ta đã biết cách tạo class đơn giản:

```
In [ ]: # Ôn tập: Class Student đơn giản từ buổi trước
class Student:
    # Class variable
    school = "ABC University"

    # Tạo object
    pass

# Tạo object
student1 = Student()
student2 = Student()

print(f"School: {Student.school}")
print(f"Student1 school: {student1.school}")
print(f"Student2 school: {student2.school}")
```

TODO 1: Ôn tập tạo Class

Tạo một class `Car` với class variable `brand = "Toyota"`. Tạo 2 object từ class này và in ra brand của chúng.

```
In [ ]: # TODO 1: Viết code của bạn ở đây
class Car:
    # Viết code ở đây
    pass

# Tạo 2 object và test
```

2. Constructor init Method

Constructor `__init__` là method đặc biệt được gọi tự động khi tạo object. Nó giúp khởi tạo các thuộc tính cho object.

```
In [ ]: class Student:
    # Class variable
    school = "ABC University"

    # Constructor - method đặc biệt
    def __init__(self, name, age):
        # Instance variables - thuộc tính riêng của mỗi object
        self.name = name
        self.age = age
        print(f"Tạo student: {name}, {age} tuổi")

# Tạo object với constructor
student1 = Student("Minh", 20)
student2 = Student("Lan", 19)

print(f"Student1: {student1.name}, {student1.age} tuổi")
print(f"Student2: {student2.name}, {student2.age} tuổi")
```

Giải thích quan trọng:

- `self` : tham chiếu đến object hiện tại
- `self.name` : thuộc tính riêng của object (instance variable)
- `__init__` : method constructor, tự động chạy khi tạo object
- Mỗi object có thể có giá trị khác nhau cho instance variables

TODO 2: Tạo Constructor

Tạo class `Book` với constructor nhận 3 tham số: `title`, `author`, `pages`. Tạo 2 object book khác nhau và in thông tin.

```
In [ ]: # TODO 2: Viết code của bạn ở đây
class Book:
    def __init__(self, title, author, pages):
        # Viết code ở đây
        pass

# Tạo 2 object book và test
```

3. Methods trong Class

Methods là các function được định nghĩa bên trong class. Chúng có thể truy cập và thay đổi thuộc tính của object.

```
In [ ]: class Student:
    school = "ABC University"

    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.grades = [] # List để lưu điểm

    # Method để giới thiệu
    def introduce(self):
        return f"Xin chào, tôi là {self.name}, {self.age} tuổi"

    # Method để thêm điểm
    def add_grade(self, grade):
        self.grades.append(grade)
        print(f"Đã thêm điểm {grade} cho {self.name}")

    # Method để tính điểm trung bình
    def get_average(self):
        if len(self.grades) == 0:
            return 0
        return sum(self.grades) / len(self.grades)

# Test methods
student = Student("Minh", 20)
print(student.introduce())

student.add_grade(8.5)
student.add_grade(9.0)
student.add_grade(7.5)

print(f"Điểm trung bình: {student.get_average()}")
```

TODO 3: Tạo Methods

Hoàn thiện class `Book` từ TODO 2 bằng cách thêm:

- Method `get_info()` : trả về thông tin sách dạng string

- Method `is_long_book()` : trả về True nếu sách có hơn 300 trang

```
In [ ]: # TODO 3: Viết code của bạn ở đây
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def get_info(self):
        # Viết code ở đây
        pass

    def is_long_book(self):
        # Viết code ở đây
        pass

# Test methods
```

4. Class Variables vs Instance Variables

Phân biệt rõ ràng giữa class variables và instance variables:

```
In [ ]: class BankAccount:
    # Class variable - chung cho tất cả object
    bank_name = "ABC Bank"
    total_accounts = 0

    def __init__(self, account_holder, initial_balance):
        # Instance variables - riêng cho từng object
        self.account_holder = account_holder
        self.balance = initial_balance

        # Tăng số lượng tài khoản
        BankAccount.total_accounts += 1

    def get_info(self):
        return f"Tài khoản: {self.account_holder}, Số dư: {self.balance}"

    @classmethod
    def get_total_accounts(cls):
        return cls.total_accounts

# Test
account1 = BankAccount("Minh", 1000000)
account2 = BankAccount("Lan", 500000)

print(f"Bank: {BankAccount.bank_name}")
print(f"Tổng số tài khoản: {BankAccount.get_total_accounts()}")
print(account1.get_info())
print(account2.get_info())
```

TODO 4: Class Variables và Instance Variables

Tạo class `Employee` với:

- Class variable: `company = "XYZ Corp"`
- Class variable: `total_employees = 0`
- Instance variables: `name`, `salary`
- Method `get_info()` : trả về thông tin nhân viên
- Tự động tăng `total_employees` khi tạo object mới

```
In [ ]: # TODO 4: Viết code của bạn ở đây
class Employee:
    # Class variables
    company = "XYZ Corp"
    total_employees = 0

    def __init__(self, name, salary):
        # Viết code ở đây
        pass

    def get_info(self):
        # Viết code ở đây
        pass

# Test với 3 nhân viên
```

5. Ví dụ thực tế: Bank Account cơ bản

Đây là prototype cho mini project chiều nay:

```
In [ ]: class BankAccount:
    bank_name = "Python Bank"

    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance
        print(f"Tạo tài khoản cho {account_holder} với số dư ban đầu: {initial_bala

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Nạp {amount}. Số dư mới: {self.balance}")
        else:
            print("Số tiền nạp phải lớn hơn 0")

    def withdraw(self, amount):
        if amount > 0 and amount <= self.balance:
            self.balance -= amount
            print(f"Rút {amount}. Số dư mới: {self.balance}")
        else:
            print("Số tiền rút không hợp lệ hoặc không đủ số dư")
```

```

def get_balance(self):
    return self.balance

def get_account_info(self):
    return f"Chủ tài khoản: {self.account_holder}, Số dư: {self.balance}"

# Demo
account = BankAccount("Nguyễn Văn A", 1000000)
print(account.get_account_info())

account.deposit(500000)
account.withdraw(200000)
print(f"Số dư hiện tại: {account.get_balance()}")

```

TODO 5: Hoàn thiện Bank Account

Thêm method `transfer(other_account, amount)` cho class `BankAccount` để chuyển tiền giữa 2 tài khoản.

```

In [ ]: # TODO 5: Copy class BankAccount ở trên và thêm method transfer
class BankAccount:
    bank_name = "Python Bank"

    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance
        print(f"Tạo tài khoản cho {account_holder} với số dư ban đầu: {initial_balance}")

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Nạp {amount}. Số dư mới: {self.balance}")
        else:
            print("Số tiền nạp phải lớn hơn 0")

    def withdraw(self, amount):
        if amount > 0 and amount <= self.balance:
            self.balance -= amount
            print(f"Rút {amount}. Số dư mới: {self.balance}")
        else:
            print("Số tiền rút không hợp lệ hoặc không đủ số dư")

    def transfer(self, other_account, amount):
        # Viết code ở đây
        pass

    def get_balance(self):
        return self.balance

    def get_account_info(self):
        return f"Chủ tài khoản: {self.account_holder}, Số dư: {self.balance}"

# Test transfer

```

```
account1 = BankAccount("Minh", 1000000)
account2 = BankAccount("Lan", 500000)
```

```
# Test chuyển tiền
```

6. Bài tập tổng hợp

TODO 6: Tạo class Product cho cửa hàng

Tạo class `Product` với:

- Instance variables: `name`, `price`, `quantity`
- Method `get_info()` : trả về thông tin sản phẩm
- Method `sell(amount)` : bán sản phẩm (giảm quantity)
- Method `restock(amount)` : nhập thêm hàng (tăng quantity)
- Method `get_total_value()` : trả về tổng giá trị hàng tồn kho

```
In [ ]: # TODO 6: Viết code của bạn ở đây
class Product:
    def __init__(self, name, price, quantity):
        # Viết code ở đây
        pass

    def get_info(self):
        # Viết code ở đây
        pass

    def sell(self, amount):
        # Viết code ở đây
        pass

    def restock(self, amount):
        # Viết code ở đây
        pass

    def get_total_value(self):
        # Viết code ở đây
        pass

# Test Product
product = Product("Laptop", 15000000, 10)
print(product.get_info())
print(f"Tổng giá trị: {product.get_total_value()}")

product.sell(3)
product.restock(5)
print(product.get_info())
```

7. Tổng kết và chuẩn bị Mini Project

Những gì đã học:

1. **Constructor** `__init__`: Khởi tạo object với thuộc tính ban đầu
2. **Methods**: Functions trong class để thao tác với object
3. **self**: Tham chiếu đến object hiện tại
4. **Class variables vs Instance variables**: Phân biệt thuộc tính chung và riêng
5. **Các method thực tế**: deposit, withdraw, transfer cho Bank Account

Chuẩn bị cho Mini Project chiều nay:

Bank Account System sẽ bao gồm:

- Class BankAccount hoàn chỉnh
- Menu cho người dùng
- Quản lý nhiều tài khoản
- Lưu trữ và hiển thị lịch sử giao dịch
- Sử dụng functions để tổ chức code

Kiến thức sẽ dùng:

- Functions (đã học tuần trước)
- Lists và Dictionaries (đã học)
- Loops và If/else (đã học)
- Input/Output (đã học)
- Classes và Methods (vừa học hôm nay)

Bài tập về nhà (Optional)

1. Tạo class `Calculator` với methods để thực hiện các phép toán cơ bản
2. Tạo class `Student` hoàn chỉnh với khả năng quản lý danh sách môn học và điểm số
3. Nghiên cứu về `__str__` method để in object đẹp hơn

Tip cho Mini Project:

- Sử dụng lists để lưu nhiều tài khoản
- Sử dụng dictionaries để lưu thông tin giao dịch
- Sử dụng functions để tạo menu và xử lý input
- Test từng method riêng biệt trước khi ghép lại